

Comparisons In AI Models For Mate-In-One Move Prediction In Chess

James Xie

dept. of System Design Engineering

University of Waterloo

Waterloo, Canada

zhiyuan.xie@uwaterloo.ca

Abstract—Creating reliable chess move predictors from models covered in the lectures is difficult due to a large, structured move space and obeying strict legal rules. I focus on supervised, masked multi-class classification of the last move from a real game chess position, as well as assessing the overall performance of the predictors. I will involve a dataset from Mate-in-One published on Kaggle, where the White side is one move from checkmate, with the predetermined best move [1]. Meanwhile, I will run a standard chess engine to obtain centipawn evaluations that serve as both regression targets and a measure of move quality. My approach will include features such as material changes, capture or promotion flags, mobility and center control changes, and king-safety indicators, along with metadata such as phase and time control. Using these features, I will train using several algorithms from lectures: logistic or ridge regression, linear and RBF-kernel SVMs, a small multi-layer perceptron, and a CNN (Convolutional Neural Network) that learns spatial patterns directly from the board tensor, with a masked softmax over all legal moves. For each model, I will perform systematic hyperparameter tuning using a training/validation split and visualize the resulting performance with model selection curves. On the held-out test CSV containing 5012 positions, linear baseline models demonstrated the strongest ranking performance. Logistic regression achieved top-1 and top-5 accuracies of 0.404 and 0.896, respectively, while ridge regression reached 0.406 and 0.889. Linear SVM produced similar results at 0.399 and 0.894. In contrast, more flexible models did not improve rank-1 selection. The RBF SVM dropped to 0.307 top-1 (0.866 top-5), and the MLP reached 0.390 top-1 (0.890 top-5), indicating that increased nonlinearity did not enhance identification of the unique mate move. The CNN underperformed in top-5 accuracy (0.693) and achieved 0.343 top-1 accuracy, suggesting that limited data and architectural capacity hindered effective learning of mating motifs from raw board tensors. In engine-based comparisons, models that failed to identify the labeled mate (Linear SVM, RBF SVM, MLP, CNN) incurred large average quality penalties relative to the CSV move, with mean centipawn gaps of 4.4×10^4 to 5.7×10^4 . This outcome is consistent with selecting substantially inferior moves when checkmate is not found. In summary, engineered move features combined with simple linear scoring proved most effective for ranking the mate-in-one solution. Increased model complexity did not result in higher top-1 accuracy under the current experimental setup. Future research should prioritize position-aware training objectives, mate-aware evaluation metrics, and scaling CNN training to a substantially larger dataset to assess whether spatial learning can reliably enhance rank-1 selection.

Index Terms—Chess move prediction, Mate-in-one, End game state, Board position evaluation, Linear regression, Support vector machines, Multi-linear perceptron, Convolutional neural network

I. INTRODUCTION

In this project, I aim to discover supervised machine learning models that predict the next move in chess positions where a forced checkmate for White exists, using regular chess positions at varying move depths.

A. Background of predicting next move in chess

Chess has been a classic testbed for AI for decades because it combines a well-defined set of rules with very high strategic complexity [2]. Advanced engines such as Stockfish and AlphaZero-style systems are trained to win games by integrating search algorithms with deep neural networks or reinforcement learning. Recently, large language models (LLMs) have also been evaluated on their ability to play chess in text form, for instance, by providing an FEN (Forsyth-Edwards Notation) string and requesting the next move to force a checkmate [2]. While these models can demonstrate notable pattern recognition, they frequently make errors such as generating illegal moves, missing straightforward tactics, or producing poorly calibrated probability estimates.

Focusing on mate-in-N positions is valuable because these tasks are central to the training and application of chess engines [3]. Modern chess platforms and applications offer millions of tactical puzzles, enabling players to practice unbiased recognition of forcing patterns efficiently without calculating positions themselves. These positions are generated by reliable engines, so a model that accurately predicts White's mating move could act as a rapid tactics recommender. Restricting the task to 'White to move and mate found by an engine' simplifies the problem to recognizing a specific, high-value tactical pattern with a clear outcome: either White wins the game or misses the opportunity.

Prior research on chess move prediction has primarily addressed general move selection rather than on mate discovery. For example, Oshri and Khandwala trained convolutional neural networks to predict human or engine moves from board images, demonstrating that CNNs can extract meaningful spatial features and achieve non-trivial top-k move accuracy [4]. Other studies have applied language models trained on PGN (Portable Game Notation) sequences to predict legal next moves, defining chess as a time-series prediction task [5]. Given that these works typically report top-5 move-matching accuracy and track how often the models suggest

illegal moves, I restrict attention to mate-in-one and white-to-move board states.

B. Dataset to be involved in the project

For this project, I use pre-prepared chess tactics datasets that are publicly available on Kaggle [1]. The two CSV files consist of engine-generated mate-in-one positions sourced from real games and composed puzzles, filtered to ensure that it is always White to move. In each case, the engine has identified a forced mate for White in exactly one move.

The input (X) is the FEN representation of each position. For each entry, I reconstruct the board from the FEN, generate all legal White moves, and construct a feature vector for each legal move. These features are numerical values that describe what happens if that move is played, for example: material change (change in White's pieces evaluation in centipawn), capture and promotion flags (binary), changes in mobility (how many legal moves White and Black after the move), check and discovered-check indicators, and king safety (changes in attack patterns around both kings). All categorical information, including piece types and squares, is converted into numerical form (e.g., through counts or lookup tables), ensuring that model inputs are numerical feature vectors rather than raw FEN strings.

The output (Y) for each position is the engine-recommended White move that achieves the shortest mate. When training, I treat it as the "correct class" among the set of legal moves for that specific position, so the model assigns the highest score to that move. This makes the problem a masked multi-class classification task: the model only scores legal White moves for the position, and the label is the index of the correct move in that list. I will split the training CSV file into a training set (about 80 %) and a validation set (about 20 %), and use this validation set for hyper-parameter tuning and model selection. The separate test CSV file is kept for final evaluation and reporting.

C. Algorithms

1) *Logistic / Ridge Regression (Linear Model)*: I will apply a linear model that scores each legal move based on its feature vector and normalizes these scores across all legal moves. The primary parameter varied is the regularization strength C (for logistic) or λ (for ridge), on a logarithm grid. I will keep other details fixed (e.g., maximum iterations, feature scaling) at reasonable values. For each value of C/λ , I will measure top-5 accuracy on the validation set (i.e., the frequency with which the model's top-5 predictions include the engine's mating move) and, when engine scores are available, the average centipawn loss for the top-1 move. This analysis will indicate the impact of regularization on both classification accuracy and move quality.

2) *SVMs*: I will implement both linear SVMs and RBF-kernel SVMs as margin-based classifiers on the move feature vectors. For the linear SVM, I will vary the penalty parameter C (on a log grid from small to large values like above) and also compare results with and without feature scaling,

while keeping other parameters fixed (e.g., loss function, max iteration). For the RBF SVM, both C and the kernel width γ will be varied over a two-dimensional grid. For each configuration, I will assess validation top-5 accuracy and present the results as a heatmap over the $C \times \gamma$ grid. On a subset of data with available engine scores, I will also evaluate how frequently the selected move preserves the mate and the corresponding centipawn loss, to determine whether higher accuracy settings correlate with improved move quality.

3) *MLP*: I will train a small feed-forward neural network that takes the move feature vector as input and outputs a score for each legal move, followed by a masked softmax. Because MLPs have many hyper-parameters, I will focus on a few that strongly affect the performance: (i) model capacity (number of hidden layers; and number of hidden units per layer); (ii) learning rate; and (iii) regularization, mainly dropout rate and weight decay [6]. I will keep the batch size at a reasonable fixed value and use early stopping based on validation loss instead of treating the number of epochs as a tuned parameter. For each configuration, I will measure training and validation top-5 accuracy, masked cross-entropy loss, and, when available, centipawn loss on a validation subset. This will provide how increasing model capacity or changing regularization affects overfitting, accuracy on mating moves, and the severity of the errors when the model chooses the wrong move.

4) *CNN*: In addition to models based on tabular features, I will implement a CNN that operates directly on the chessboard representation. For each position, I will convert the FEN into an $8 \times 8 \times C$ tensor of piece planes, which serves as input to a compact convolutional network designed to learn spatial patterns such as mating motifs, piece coordination, and king exposure. The final layers of the CNN will output a vector of scores over a fixed move-encoding space. During both training and testing, I will apply softmax to the legal White moves, ensuring correct model behavior.

For the CNN, I will vary several key parameters that significantly influence performance: (i) the number of convolutional layers, (ii) the number of filters per layer, and (iii) the learning rate [4]. Other parameters, such as batch size and the number of training epochs, will remain fixed, with early stopping based on validation loss employed instead of tuning these values. For each CNN configuration, I will assess top-5 accuracy in predicting the engine's mating move on the validation set and, when engine scores are available, the average centipawn loss after executing the model's top-1 move. Comparing these outcomes with those from linear models and MLPs will determine whether direct learning of spatial features from the board provides a substantial advantage in mate-in-one positions.

II. EXPERIMENTS

Based on the mate-in-one Kaggle dataset and the model families introduced previously, I will systematically describe the procedures for feature construction, data splitting, model training, and performance evaluation to ensure comparability across all models.

A. Data Preprocessing and Feature Construction

For each FEN, I reconstruct the board state (piece placement, side to move, castling rights, en-passant square, half-move clock, and move number) and generate the full set of legal moves for White. For tabular models (logistic/ridge regression, SVMs, MLP), I construct a fixed-length feature vector for each legal move. The features considered comprise material change (measured in centipawns), binary indicators for captures and promotions, flags for whether the move results in check, discovered check, or double check, as well as king-safety features, for example, the change in the number of attacks on squares adjacent to each king. All categorical variables are converted to numerical form using one-hot encoding or small lookup tables, resulting in a numerical feature vector for every move.

For the CNN experiments, I further encode each position as an $8 \times 8 \times C$ tensor, where each plane corresponds to the presence of a specific piece type for each color. Additional planes are used to record the side to move and castling rights, thereby capturing all relevant aspects of the board state.

B. Train/Validation/Test Splits

To support fair comparison across models, I adopt a position-based split: Training set ($\sim 80\%$) used to fit model parameters; Validation set ($\sim 20\%$) used for hyperparameter tuning and early stopping; and Test set (a separate CSV file, is held out entirely for final evaluation.

All legal moves corresponding to a particular position are assigned to the same data split. For each position, the label is defined as the index of the engine-identified mating move within the set of legal moves. As a result, the task becomes a masked multi-class classification problem, where the number of classes varies by position but is always limited to the legal moves available in that position.

C. Model training and hyperparameter tuning

All models are trained using a consistent move-level feature representation for the regression models, SVMs, and MLP, or the $8 \times 8 \times C$ tensor for the CNN. Hyperparameter tuning is conducted via group-aware cross-validation over positions, and model selection is primarily determined by the top-5 prediction accuracy on the validation set.

For the regression models, I tune the regularization strength over a logarithmic grid $C \in \{10^{-3}, \dots, 10^2\}$ for logistic regression and $\lambda \in \{10^{-3}, \dots, 10^2\}$ for ridge regression. In the case of SVMs, I vary C over a similar grid for the linear kernel ($10^{-3}, 10^{-2}, \dots, 10^3$), and for the RBF kernel, I employ a two-dimensional grid with $C \in \{10^{-4}, \dots, 10^1\}$ and $\gamma \in \{10^{-1}, \dots, 10^3\}$. For the MLP, the hyperparameter search is divided into two stages. In the first stage, I fix the learning rate and set dropout rates and weight decay to zero, while varying the number of hidden layers and the number of hidden units per layer in each experiment ($[32], [64], [128], [256], [32, 32], [32, 64], [32, 128], [32, 256], \dots, [256, 32], [256, 64], [256, 128], [256, 256]$). After completing stage one, I select the best-performing configuration to

further investigate the effects of varying learning rates ($10^{-3}, 10^{-2}, 10^{-1}, 1$), dropout rates (0.0, 0.1), and weight decay (0, 10^{-4}) in the second stage.

For the CNN, I implement a model that operates directly on an 8×8 board representation, supplemented by an explicit encoding of the candidate move. For each legal move in a given position, I construct a stack of 19 planes: 17 planes encode the board state, including piece locations and side to move, while 2 additional planes indicate the from-square and to-square of the candidate move [8]. The resulting tensor has shape $X_{cnn} \in \mathbb{R}^{N \times 19 \times 8 \times 8}$, where $N = 38001$ moves are sampled from the first 1000 positions in the training set. Each move is assigned a binary label $y \in \{0, 1\}^N$ to indicate whether it is the engine’s mate-in-one move, and `pos_id` is used to track which moves correspond to the same position.

To explore architecture capacity, I defined a grid of 20 configurations. First, I considered single-layer convolutional networks with widths $\{32, 64, 128, 256\}$. Subsequently, I include all two-layer combinations of these widths, such as $(32, 32), (32, 64), \dots, (256, 256)$. In total, this results in 4 single-layer and 16 two-layer configurations, yielding 20 distinct settings.

D. Evaluation metrics

The primary evaluation metric is the positional-level top-k move-matching accuracy. Top-5 accuracy measures the proportion of positions in which the correct move is included among the five highest-scoring legal moves. This metric is particularly relevant because practical decision support systems may present multiple candidate moves, and prior research on chess prediction commonly reports top-k metrics. Therefore, top-5 accuracy serves as the principal criterion for model selection and comparison [5].

For each model family and hyperparameter configuration, I report the mean top-5 accuracy across validation splits during model selection, as well as the corresponding top-5 accuracy on the held-out test positions for the final selected configuration in each family. In addition, qualitative error analysis is conducted by examining example positions where the model fails to include the correct mating move in its top-5 predictions and by identifying common tactical patterns that are missed.

The final evaluation is conducted on the separate held-out test CSV provided [1]. Two categories of metrics are computed. First, move-matching accuracy is assessed relative to the ground-truth move specified in the test CSV. Top-1 accuracy is defined as the proportion of test positions for which the highest-ranked move exactly matches the labeled move.

Second, to further quantify move quality beyond exact label matching, I perform a Stockfish-based centipawn gap analysis, using the CSV label as the reference move. For each position, Stockfish evaluates the resulting position after playing (i) the CSV-labeled move and (ii) the model’s top-1 move, using a fixed search depth and reporting evaluation scores from the perspective of the original side to move.

III. RESULTS

A. Regression

1) *Logistic Regression*: The model-selection curve indicates that validation top-5 accuracy is relatively stable across the explored C values, with small split-to-split variability. For the selected $C = 0.01$, as shown in Figure 1 below, the printed split results show mean train top-5 accuracy of 0.8993 ± 0.0003 , and validation top-5 accuracy of 0.9003 ± 0.0013 .

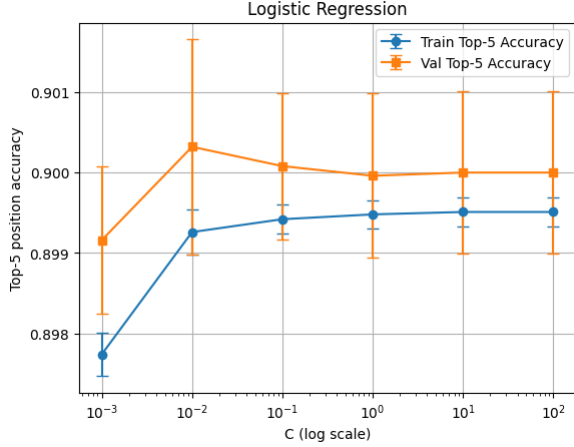


Fig. 1. Logistic Regression Model Selection Curve

Under the test-set evaluation protocol, logistic regression achieved a top-1 accuracy of 0.4044 and a top-5 accuracy of 0.8957. This result suggests that the CSV-labeled mating move is frequently ranked among the top candidates, even when not ranked first. In the Stockfish-based move-quality analysis, comparison of the model's top-1 move with the CSV-labeled move yielded a mean signed centipawn gap $\Delta_{cp} = cp_{CSV} - cp_{model}$ of -64122.08 cp, with a 95% confidence interval of $[-65686.46, -62557.70]$. This finding indicates that Stockfish evaluated the model's selected move as stronger than the CSV-labeled move on average. Furthermore, the proportion of positions with $\Delta_{cp} < 0$ (indicating the model move was evaluated as better) is 0.596.

2) *Ridge Regression*: Ridge classification is evaluated using the grouped 5-split protocol and position-wise top-5 accuracy metric. For model selection, the regularization parameter λ is varied over a logarithmic scale from 10^{-3} to 10^2 in six steps. The model selection curve remained nearly flat, indicating that performance is largely insensitive to λ within the tested range. For instance, at $\lambda = 0.001$, the split-wise results yield a mean training top-5 accuracy of 0.8981 ± 0.0003 and a mean validation top-5 accuracy of 0.8984 ± 0.0013 . The hyperparameter $\lambda = 0.001$ is selected, as shown in Figure 2 on the top right, although this choice reflects only marginal differences rather than a distinct optimum.

On the held-out test set, the ridge classifier achieves a top-1 accuracy of 0.4064 and a top-5 accuracy of 0.8887. These results indicate that, for most positions, the CSV-labeled mating move is assigned a high rank, frequently

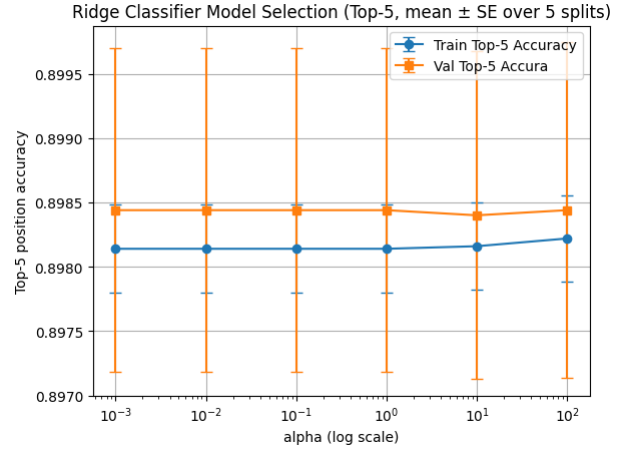


Fig. 2. Ridge Regression Model Selection Curve

within the top-5, while exact agreement at rank 1 occurs in approximately 41% of cases. The signed centipawn gap is calculated as $\Delta_{cp} = cp_{CSV} - cp_{model}$, with a mean centipawn gap of -75026.27 and a 95% confidence interval of $[-76988.60, -73063.94]$. This outcome suggests that, under this scoring protocol, Stockfish rated the model's top-1 move higher than the CSV-labeled move in 59.2

B. SVMs

1) *Linear SVM*: A linear SVM (Linear SVC) is tuned over $C \in \{10^{-3}, 10^{-2}, \dots, 10^3\}$ using five grouped splits. Evaluation employed position-wise top-5 accuracy, determined by ranking legal moves within each position according to the SVM decision scores. Performance remains largely consistent across the tested C values. The hyperparameter $C = 1$ is selected based on its highest mean validation top-5 accuracy across splits, as illustrated in Figure 3, the mean training top-5 accuracy is given by 0.8999 ± 0.0001 , while mean validation top-5 accuracy is 0.9016 ± 0.0009 .

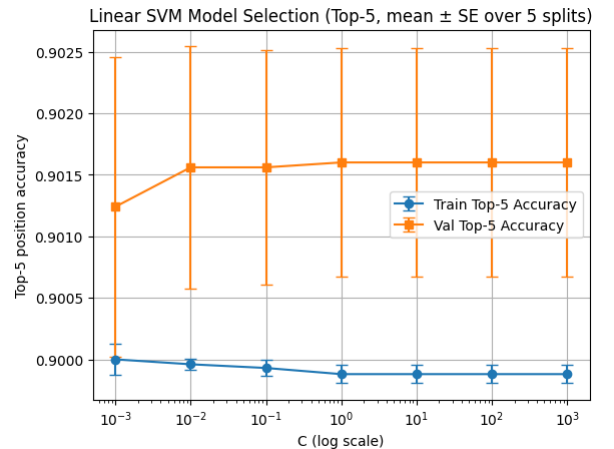


Fig. 3. Linear SVM Model Selection Curve

On the held-out test set, the linear SVM reaches a top-1 accuracy of 0.3992 and a top-5 accuracy of 0.8939. This result indicates that the CSV-labeled mating move is typically ranked within the top candidate set, although exact top-1 agreement occurs in approximately 40% of the positions. In the Stockfish-based move-quality analysis, which compares the model’s top-1 move to the CSV-labeled move, the mean signed centipawn gap Δ_{cp} is 44107.68 with a 95% confidence interval of [42729.71, 45485.64]. This finding suggests that Stockfish evaluated the model’s selected move as substantially worse than the CSV move. Consistent with this result, the fraction of positions with $\Delta_{cp} < 0$ is 0.

2) *RBF SVM*: A two-dimensional grid search is conducted over $C \in \{0.1, 1, 10, 100, 1000\}$ and $\gamma \in \{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1, 10\}$ (both in logspace), with hyperparameters selected based on mean validation top-5 accuracy averaged over group-based splits. The validation surface exhibits a distinct peak at moderate γ and large C . The optimal configuration is $C = 1000, \gamma = 1$, which achieves a top-5 accuracy of 0.8655 ± 0.0013 . A nearby setting ($C = 100, \gamma = 1$) yields a slightly lower accuracy of 0.8581 ± 0.0021 . When γ is increased to values such as $\gamma = 10$, performance declines to near chance-level, with a top-5 accuracy of ≈ 0.32 . This result is consistent with severe overfitting and instability in the decision function, as illustrated in Figure 4.

For positions in the held-out test CSV, the RBF-kernel SVM achieves a top-1 accuracy of 0.3065 and a top-5 accuracy of 0.8655. The model generally ranks the CSV-labeled move highly, but selects it as the top choice in approximately 30% of cases. To evaluate move quality, Stockfish assessments are performed after applying the model’s top move, and the signed centipawn gap Δ_{cp} is averaged across positions. The mean centipawn gap is 52518.65, with a 95% confidence interval of [51126.45, 53910.86], indicating a substantial decrease in engine-evaluated strength when the model’s top-1 move is chosen instead of the CSV move. As with the linear SVM, the proportion of positions where $\Delta_{cp} < 0$ remains 0.

C. MLP

The MLP model selection process is divided into two stages. Stage 1 focuses on architecture search, while Stage 2 addresses regularization and learning rate optimization.

In Stage 1, 20 architectures (1–2 hidden layers; widths {32, 64, 128, 256}) are evaluated using five GroupShuffleSplit folds and position-wise top-5 accuracy. The best-performing architecture is (256,128), with a mean validation top-5 accuracy of 0.8765 ± 0.0017 . The next-best architecture, (256,64), achieves the same mean but exhibits higher variability across splits. Figure 5 presents the model selection curve for the evaluated architectures.

Subsequently, with the architecture fixed at (256,128), a grid search is conducted over dropout $\in \{0, 0.1, 0.2\}$, weight decay $\in \{0, 10^{-4}\}$, and learning rate $\in \{10^{-3}, 10^{-2}, 10^{-1}, 1\}$ using three GroupShuffleSplit folds and validation top-5 accuracy. The optimal configuration is dropout = 0.2, weight decay = 0, and learning rate = 0.01, resulting in a mean validation

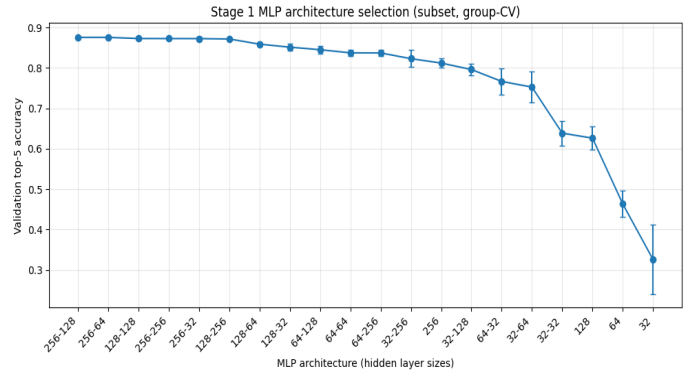


Fig. 4. MLP Model Selection Curve - Stage 1

top-5 accuracy of 0.9131 ± 0.0011 , as shown in Table 1 (see Appendix A).

During the testing phase, the MLP achieves a top-1 accuracy of 0.3901 and a top-5 accuracy of 0.8901. This result indicates that the CSV-labeled move is generally ranked near the top of the candidate list, although it is selected as the highest-ranked move in less than 40% of cases. In the Stockfish-based move-quality analysis, the mean signed centipawn gap is 56453.64, with a 95% confidence interval of [55086.64, 57820.65]. This finding suggests that Stockfish evaluates the model’s chosen move as substantially worse than the CSV move. Additionally, the $\Delta_{cp} < 0$ case does not occur in the MLP results.

D. CNN

To evaluate whether spatial features improve mate-in-one prediction, a small CNN is trained on the board-plus-candidate-move tensor (19 planes). An architecture sweep is conducted over 20 convolutional configurations, including single-layer widths of 32, 64, 128, and 256, as well as all two-layer combinations. Five group-aware splits are used, and position-wise top-5 accuracy with associated uncertainty was reported across splits.

The model selection curve remains relatively flat, with most architectures achieving top-5 accuracies in the mid-0.30s range. Adding a second convolutional layer does not consistently improve validation performance. For instance, larger two-layer models such as (256,64) and (256,128) achieve mean validation top-5 accuracies of 0.3460 and 0.3470, respectively, while (256,256) reaches 0.3580.

The best-performing configuration consists of a single convolutional layer with 128 channels, achieving the highest mean validation top-5 accuracy of 0.3860 among the tested architectures. This result is narrowly ahead of several close alternatives, including a single 256-channel layer and (64, 32) layers, as shown in Figure 6 (See Appendix B).

Over the held-out testing, the CNN achieves a top-1 accuracy of 0.3430 and a top-5 accuracy of 0.6927, demonstrating substantially weaker recovery of the CSV-labeled mode among the top-ranked candidates compared to the tabular models. In the Stockfish-based move-quality analysis, which compares the model’s top-1 move to the CSV-labeled move,

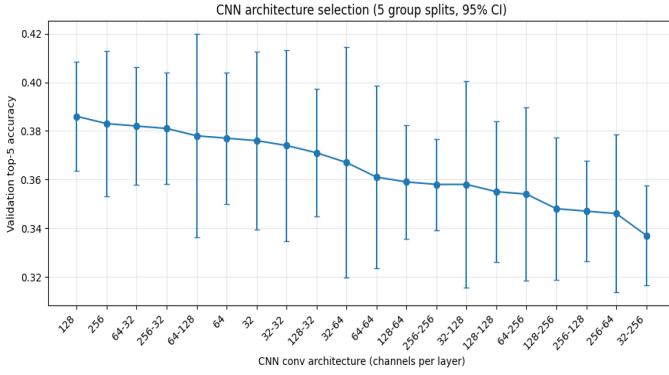


Fig. 5. MLP Model Selection Curve - Stage 1

the mean signed centipawn gap Δ_{cp} was 56718.45, with a 95% confidence interval of [55280.81, 58156.09]. This result indicates that the model’s selected move is generally evaluated as significantly worse than the CSV move. Furthermore, there were no instances where $\Delta_{cp} < 0$, which further supports this conclusion.

IV. DISCUSSION

A. Summary of empirical patterns

A consistent ranking pattern was observed across all approaches: top-5 accuracy generally exceeded top-1 accuracy. The linear baselines, specifically logistic regression and ridge regression, achieved the highest top-5 performance (both near 0.89 to 0.90), while their top-1 accuracy remained moderate (approximately 0.40). The linear SVM demonstrated comparable top-5 accuracy and slightly lower top-1 accuracy. In contrast, the RBF SVM and the CNN exhibited notably weaker top-1 accuracy (RBF approximately 0.31; CNN approximately 0.34). The CNN, in particular, showed a substantial decrease in top-5 accuracy (approximately 0.69). These results suggest that most models are able to assign a relatively high rank to the labeled move, yet many do not consistently identify it as the highest-scoring candidate. This indicates that distinguishing the true mate from near-miss moves within a given position remains the primary challenge.

B. Interpretation of centipawn gaps among different models

Centipawn gaps are computed as $\Delta_{cp} = cp_{CSV} - cp_{model}$, where a positive value indicates the model-selected move is worse than the CSV-labeled mate move under fixed Stockfish scoring. For Linear SVM, RBF SVM, MLP, and CNN, Δ_{cp} is large and strictly positive on average (with $\Pr(\Delta_{cp} < 0) = 0$), consistent with the expectation that failing to play the mate-in-one move typically leads to a materially inferior position. In these models, the mean gap essentially coincides with the mean clipped loss $\max(0, \Delta_{cp})$, implying that the model’s top-1 choice was almost never evaluated as superior to the labeled solution.

By contrast, logistic regression and ridge regression produced negative mean gaps in the reported runs, and $\Pr(\Delta_{cp} <$

0) ≈ 0.59 –0.60, which implies Stockfish frequently rate the model’s move above the CSV-labeled move. Under a dataset claim that each position has a single mate-in-one solution, this pattern is unlikely to reflect genuine superiority of the models and instead suggests a systematic evaluation mismatch affecting the CSV move’s centipawn score in those runs. In particular, mate-in-one positions are highly sensitive to mate-score handling (terminal checkmate detection, mate-to-centipawn conversion, and depth-dependent mate reporting). If the CSV move’s post-move position is a terminal checkmate but is not consistently mapped to a maximal positive score, Δ_{cp} can be biased negative even when the model move is objectively worse. Given that the later models exhibit the expected strictly positive gaps under the same protocol, the most defensible interpretation is that the centipawn-gap metric is informative for comparing how severely a model degrades from the mate move only when mate handling is fully consistent across all models and runs.

C. Future work

Several targeted modifications follow directly from the observed ranking and centipawn patterns:

Firstly, we can optimize for rank-1 decisions with a position-aware objective. Replace independent binary scoring with a masked softmax (per-position cross-entropy over legal moves) or a listwise ranking loss. This directly addresses the persistent top-5 vs top-1 gap by enforcing stronger within-position separation.

Moreover, adding hard-negative mining within positions can improve the top-ranked move selection. During training, up-weight the most competitive incorrect moves (highest-scoring distractors) since these are exactly the errors that preserve top-5 performance while harming top-1 accuracy.

Lastly, featuring refinements can be helpful for linear models. Add mate-specific discriminators (e.g., king escape-square coverage, checking attacker count, pinned defender indicators) to improve rank-1 calibration while preserving the robustness of linear scoring.

D. Conclusion

Overall, the models demonstrated strong ability to *surface* the labeled mate move among a small candidate set (high top-5), but weaker ability to *consistently select* it as the top choice (moderate top-1). Centipawn gaps for several models confirm that missing the mate typically incurs a large quality penalty, while anomalous negative gaps highlight the need for strictly consistent mate-aware evaluation when using engine-based metrics.

REFERENCES

- [1] Bhupen, “Mate in One (Chess),” Kaggle.com, 2019. <https://www.kaggle.com/datasets/ancientaxe/mate-in-one-chess> (accessed Nov. 27, 2025).
- [2] K. B. Diallo and M. A. Akhloufi, “ChessMoveLLM: Large Language Models for Chess Next Move Prediction,” *SoutheastCon*, 2025, Concord, NC, USA, 2025, pp. 475-480, doi: 10.1109/Southeast-Con56624.2025.10971611.

- [3] D. Brumleve, J. D. Hamkins, and P. Schlicht, "The mate-in-n problem of infinite chess is decidable," *arXiv.org*, 2025. <https://arxiv.org/abs/1201.5597>
- [4] B. Oshri and N. Khandwala, "Predicting Moves in Chess using Convolutional Neural Networks," 2015. Available: <https://cs231n.stanford.edu/reports/2015/pdfs/ConvChess.pdf>
- [5] A. Stöckl, "Watching a Language Model Learning Chess," *Proceedings of the Conference Recent Advances in Natural Language Processing - Deep Learning for Natural Language Processing Methods and Applications*, pp. 1369–1379, Sep. 2021, doi: <https://aclanthology.org/2021.ranlp-1.153/>.
- [6] M. F. Arsalan and H. Soeparno, "Deep Learning Approaches to Predicting the Optimal Chess Moves from Board Positions," *International Journal of Engineering Trends and Technology*, vol. 72, no. 4, pp. 43–50, Apr. 2024, doi: <https://doi.org/10.14445/22315381/ijett-v72i4p105>.
- [7] M. Sabatelli, F. Bidoia, V. Codreanu, and M. Wiering, "Learning to Evaluate Chess Positions with Deep Neural Networks and Limited Lookahead," *Proceedings of the 7th International Conference on Pattern Recognition Applications and Methods*, 2018, doi: <https://doi.org/10.5220/0006535502760283>.
- [8] M. F. Arsalan and H. Soeparno, "Deep Learning Approaches to Predicting the Optimal Chess Moves from Board Positions," *International Journal of Engineering Trends and Technology*, vol. 72, no. 4, pp. 43–50, Apr. 2024, doi: <https://doi.org/10.14445/22315381/ijett-v72i4p105>.
- [9] Bengal1, "GitHub - Bengal1/Simple-CNN-Guide: A beginner's guide to building Convolutional Neural Networks (CNNs).," GitHub, 2025. <https://github.com/Bengal1/Simple-CNN-Guide> (accessed Dec. 6, 2025).

ACKNOWLEDGMENT

A portion of the CNN model implementation (SimpleCNN) was developed in Jupyter Notebook with reference to [9].

ChatGPT was used to assist with English grammar checking and code debugging. All final wording, code changes, and technical judgments were made by the authors, and outputs were verified.

V. APPENDICES

APPENDIX A HYPERPARAMETER SEARCH RESULTS

TABLE I
TOP VALIDATION TOP-5 PERFORMANCE ACROSS SELECTED HYPERPARAMETER SETTINGS.

Run ID	hidden_sizes	dropout	weight_decay	lr	val_top5_mean	val_top5_se
17	[256, 128]	0.2	0.0000	0.01	0.913133	0.001122
1	[256, 128]	0.0	0.0000	0.01	0.911867	0.001338
9	[256, 128]	0.1	0.0000	0.01	0.910333	0.002562
21	[256, 128]	0.2	0.0001	0.01	0.909133	0.003806
5	[256, 128]	0.0	0.0001	0.01	0.909133	0.002954

APPENDIX B

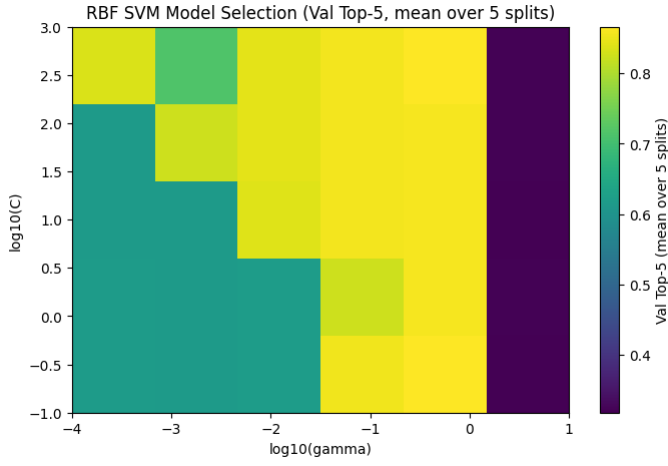


Fig. 6. RBF-kernel SVM Model Selection Curve